



Universidad Nacional de Costa Rica
Sede Regional Brunca
Campus PZ - Coto

Cátedra Ingeniería en Sistemas
Bachillerato en Ingeniería en Sistemas de Información

Estudiantes:

Francisco Mora Cabezas
Ángel Segura Méndez

Índice general

1	Introducción	7
2	Antecedentes	8
3	Descripción del Proyecto	9
4	Planteamiento del Problema	12
5	Justificación	13
5.1	Justificación e importancia	13
6	Objetivos.....	14
6.1	Objetivo general.....	14
6.2	Objetivos específicos	14
7	Metodología	16
7.0.1	Fundamentación	16
7.0.2	Referencia al WBS	16
7.0.3	Aplicación al proyecto	17
8	Estándares	19
8.1	Estándares de Documentación	19
8.2	Estándares de Programación	19
8.3	Estándares de Base de Datos	20
8.4	Estándares de Interfaz.....	20

9 Costos y Factibilidad Económica	21
9.1 3.3.3. Factibilidad Económica	21
9.2 Costos de desarrollo	21
9.3 Costos de capital humano	21
9.4 Cargos administrativos	22
9.5 Costos totales del proyecto	22
9.6 Resumen consolidado de costos	22
9.7 Conclusiones de la factibilidad económica	24
10 Desarrollo	25
10.1 Introducción	25
10.2 Gestión de Reuniones y Documentación	25
10.2.1 Formato de Minutas de Reunión	25
10.2.2 Gestión de Cambios	26
10.3 Arquitectura Técnica	27
10.3.1 Stack Tecnológico	27
10.3.2 Arquitectura Monorepo	28
10.4 Módulo de Evidencias SINAES	28
10.4.1 Descripción y Cliente	28
10.4.2 Componentes Principales	28
10.4.3 Estructura de Backend	29
10.4.4 Interfaz de Usuario	29
10.5 Módulo de Tiempos de Jornada	29
10.5.1 Descripción y Cliente	29
10.5.2 Componentes Principales	30
10.5.3 Modelo de Datos	30
10.5.4 Estructura de Backend	30
10.5.5 Interfaz de Usuario	31
10.6 Integración entre Módulos	31
10.7 Estado Actual del Desarrollo	32
10.8 Implementación y Despliegue	32
10.8.1 Plan de Implementación del Sistema	32
10.8.2 Estrategia de Despliegue y Ambientes Definidos	33
10.8.3 Procedimiento de Instalación y Configuración	33
10.8.4 Plan de Respaldos y Recuperación	34
10.8.5 Capacitación al Usuario Final y Entrega de Manuales	35
10.9 Aceptación por Parte del Usuario	35
10.9.1 Plan de Pruebas de Aceptación del Usuario (UAT)	35

10.9.2	Ejecución y Evidencias de las Pruebas	35
10.9.3	Bitácora de Hallazgos y Correcciones Aplicadas	36
10.9.4	Acta de Aceptación	36
10.10	Desafíos	36
10.10.1	Desafíos Técnicos Enfrentados	36
10.10.2	Riesgos Materializados y Acciones de Mitigación	37
10.10.3	Cambios de Alcance y su Impacto	37
10.10.4	Lecciones Aprendidas	37
10.11	Detalles Técnicos	38
10.11.1	Arquitectura del Sistema	38
10.11.2	Tecnologías, Lenguajes y Frameworks Utilizados	38
10.11.3	Repositorio del Código Fuente y Control de Versiones	39
10.11.4	Estándares de Codificación Aplicados	39
10.11.5	Requerimientos de Hardware y Software para la Ejecución	39
10.11.6	Seguridad: Autenticación, Autorización y Manejo de Datos Sensibles	40
11	Conclusiones	42
11.1	Logros hasta la Fecha (Fase de Planificación)	43
12	Recomendaciones	44
12.0.1	Enfoque Técnico y Arquitectónico	44
12.0.2	Estrategias Operativas y de Adopción	45
12.0.3	Crecimiento y Alcance Futuro	46
12.0.4	Gestión del Conocimiento y Documentación	47
12.0.5	Sostenibilidad a Largo Plazo	48
A	Anexos	49
	Anexo 1: Carta de presentación	49
	Anexo 2: Carta de intenciones	49
	Anexo 3: Código de Ética	49
	Anexo 4: Propuesta del proyecto	49
	Anexo 5: Registro de actores del proyecto	49
	Anexo 6: Formato de minutas de reunión	49
	Anexo 7: WBS (Estructura de Desglose de Trabajo)	49
	Anexo 8: SRS	50
	Anexo 9: Modelo No Relacional	50
	Anexo 10: Diccionario de Datos (general)	50
	Anexo 11: Formulario de gestión de cambios	50
	Anexo 12: Carpeta de firmas	50
	Anexo 13: Desarrollo	50
	Anexo 14: Diccionario de Datos — Evidencias SINAES	50
	Anexo 15: Diccionario de Datos — Tiempos de Jornada	50

Anexo 16: Glosario	50
Anexo 17: Análisis	50
Anexo 18: Diseño (diagramas)	50
Referencias bibliográficas	52

Índice de figuras

7.1	Estructura de Desglose del Trabajo (WBS) del proyecto.	17
A.1	Cronograma de Sprints del Proyecto Ingeniería en Sistemas II. . . .	51

Índice de tablas

9.1	Consolidado General de Costos del Proyecto	23
-----	--	----

1. Introducción

La Universidad Nacional de Costa Rica (UNA) busca modernizar y optimizar sus sistemas de gestión, especialmente aquellos relacionados con la calidad académica y la administración de recursos. En este contexto, la universidad enfrenta desafíos significativos en dos áreas clave: por un lado, la gestión de evidencias para los procesos de acreditación ante el Sistema Nacional de Acreditación de la Educación Superior (SINAES), y por otro, la administración eficiente de los tiempos de jornada universitaria, un recurso vital para la docencia y los proyectos institucionales.

Actualmente, la gestión de evidencias SINAES se realiza de manera manual y se apoya en una estructura de almacenamiento basada en carpetas en Google Drive. Este enfoque genera ineficiencias significativas, como la redundancia documental, una clasificación manual que consume un promedio de dos horas diarias por coordinador, y una marcada dificultad para correlacionar las evidencias con los criterios específicos de calidad definidos por SINAES. El problema principal radica en la ausencia de un sistema automatizado que permita una categorización eficiente y la falta de metadatos asociados a los documentos, lo que resulta en una dependencia de criterios individuales para la clasificación. Estas deficiencias no solo conllevan un desperdicio de recursos institucionales y la duplicación de la memoria utilizada, sino que también incrementan el riesgo de errores en la verificación del cumplimiento de criterios y pueden generar retrasos críticos en la preparación de informes para auditorías SINAES. Además, la falta de seguridad adecuada para los documentos institucionales representa un riesgo potencial de pérdida de acreditación en carreras clave. Para mitigar estos desafíos, se propone el desarrollo de un módulo de gestión de evidencias como parte del macroproyecto "Gestión de Calidad", el cual estará enfocado exclusivamente en la interfaz de usuario para la manipulación directa de documentos en Google Drive, sin una capa de backend dedicada en esta fase inicial.

2. Antecedentes

La Universidad Nacional (UNA) es una institución pública de educación superior costarricense, comprometida con la excelencia académica y la pertinencia social de sus programas formativos. Desde su fundación en 1973, la UNA ha impulsado procesos de evaluación y mejora continua con el propósito de garantizar la calidad de la educación superior en el país [UNA, 2025].

En el ámbito de la acreditación, la Universidad participa activamente en los procesos regulados por el Sistema Nacional de Acreditación de la Educación Superior (SINAES), organismo que establece estándares de calidad a partir de dimensiones, componentes y criterios específicos [SIN, 2025]. Este marco exige a las carreras universitarias disponer de mecanismos formales para recopilar, organizar y reportar evidencias que sustenten el cumplimiento de dichos criterios.

Asimismo, la UNA administra de forma permanente recursos humanos y académicos a través de los denominados tiempos de jornada, los cuales representan unidades de planificación y presupuesto destinadas a la asignación de labores docentes y de proyectos institucionales. La gestión de este recurso resulta estratégica para garantizar la equidad en la distribución de la carga académica y la ejecución eficiente de iniciativas de investigación y extensión.

La revisión de documentos institucionales y normativas evidencia la necesidad de fortalecer los procesos administrativos y académicos mediante herramientas tecnológicas que permitan centralizar, automatizar y resguardar la información relacionada con acreditación y planificación de tiempos de jornada [Reg, 2024, Inf, 2024].

3. Descripción del Proyecto

El proyecto se enmarca dentro del macro-proyecto denominado “Gestión de Calidad”, cuyo propósito es apoyar a la Universidad Nacional en la optimización de procesos administrativos y académicos mediante la digitalización y automatización de la información clave. Este esfuerzo responde a la necesidad institucional de contar con sistemas más confiables, eficientes y seguros para la gestión de procesos vinculados con la acreditación académica y la distribución de recursos humanos y académicos.

De forma particular, se desarrollan dos módulos principales:

Sistema de Gestión de Evidencias para Acreditación SINAES

El Sistema de Gestión de Evidencias tiene como objetivo atender las debilidades actuales en la administración de documentos requeridos por los procesos de acreditación. Actualmente, la gestión se realiza mediante carpetas en Google Drive, lo que genera duplicidad, pérdida de tiempo en la búsqueda, falta de metadatos estandarizados y riesgo de inconsistencias en los informes.

El módulo propuesto introduce una estructura jerárquica de navegación (dimensión → componente → criterio) alineada con los lineamientos del SINAES, garantizando que cada evidencia se ubique en el criterio que le corresponde. Asimismo, cada documento se enriquece con metadatos normalizados (tipo, responsable, fecha, carrera, periodo), lo que permite búsquedas rápidas y generación de reportes confiables.

Funcionalidades principales

- Registro de evidencias con validación de formatos (PDF, DOCX, XLSX, PNG).
- Asignación de metadatos obligatorios y opcionales según el criterio SINAES.
- Búsqueda avanzada y filtrado por carrera, criterio, estado de cumplimiento o fecha.
- Generación de reportes en PDF con tablas comparativas por carrera o facultad.
- Registro de auditoría de accesos y modificaciones.

Beneficios esperados

- Reducción del tiempo de clasificación manual en al menos un 70%.
- Mejora en la trazabilidad documental y en la preparación de auditorías.
- Aseguramiento de la integridad y centralización de los documentos institucionales.
- Disminución del riesgo de pérdida de acreditaciones por falta de evidencias confiables.

Alcances y limitaciones

En esta primera fase, el sistema se concentrará en la interfaz de usuario y en la integración con Google Drive como repositorio oficial. No se incluye aún un backend propio para la gestión de evidencias, dado que el objetivo es entregar un prototipo funcional que demuestre la factibilidad y utilidad del modelo planteado.

Sistema de Gestión de Tiempos de Jornada Universitarios

El Sistema de Tiempos de Jornada atiende la necesidad de distribuir de manera justa y transparente la carga docente y los recursos destinados a proyectos institucionales. Actualmente, la administración se realiza con hojas de cálculo que dificultan la consolidación de información, generan errores de cálculo y limitan la capacidad de generar reportes confiables.

Este módulo permitirá calcular automáticamente las horas de jornada según las mallas curriculares, los incrementos por repitencia y los tiempos otorgados por convenios externos. Además, se registrarán proyectos institucionales con consumo de tiempo docente, lo que aportará una visión integral de la utilización de este recurso.

Funcionalidades principales

- CRUD de catálogos (carreras, cursos, mallas curriculares, docentes y grupos).
- Registro y actualización de cargas docentes por periodo académico.
- Gestión de proyectos institucionales con asignación de horas.
- Reglas de cálculo para contemplar repitencia y tiempos externos (fijos e inciertos).
- Exportes de reportes en Excel por ciclo, carrera, sede y comparativos históricos.

Beneficios esperados

- Transparencia en la asignación de carga académica entre docentes.
- Optimización en el uso del presupuesto destinado a tiempos de jornada.
- Reducción de errores de cálculo por el uso de herramientas automatizadas.

- Base de datos centralizada que facilita la generación de reportes estratégicos.

Alcances y limitaciones

El sistema se desarrollará con una base de datos no relacional en MongoDB, administrada a través de un backend en NestJS con Prisma ORM. En la presente etapa no se contempla la integración con otros sistemas de la universidad, aunque la arquitectura se diseñará modular para permitir futuras expansiones.

En conjunto, ambos módulos fortalecen la capacidad institucional de planificación estratégica, garantizando mayor eficiencia, transparencia y confiabilidad en la gestión académica y administrativa de la UNA. Además, constituyen un primer paso hacia la construcción de un ecosistema digital integral para la gestión de la calidad en la universidad.

4. Planteamiento del Problema

En la Universidad Nacional, los procesos de gestión de evidencias para la acreditación de carreras presentan dificultades asociadas con la dispersión, duplicidad y falta de trazabilidad de los documentos institucionales. Estas limitaciones provocan retrasos en la preparación de informes para los procesos de acreditación del SINAES y aumentan el riesgo de incumplimiento de los estándares de calidad exigidos.

De forma paralela, la administración de los tiempos de jornada universitarios se lleva a cabo mediante procedimientos manuales o apoyados en hojas de cálculo, lo cual genera inconsistencias en el cálculo de cargas docentes, dificultades en la planificación de proyectos institucionales y una limitada capacidad para generar reportes confiables en el corto plazo.

Esta situación refleja la ausencia de un mecanismo tecnológico unificado que permita centralizar y automatizar la gestión de evidencias y de tiempos de jornada. Dicha carencia afecta directamente la eficiencia administrativa, la transparencia en la toma de decisiones y la capacidad institucional para responder a los procesos de acreditación y planificación académica.

En consecuencia, se identifica como problema la necesidad de implementar un sistema web integrado que atienda estas deficiencias, facilite el cumplimiento de estándares de calidad y fortalezca la gestión estratégica de la Universidad Nacional.

5. Justificación

5.1. Justificación e importancia

La implementación de estos sistemas es crucial para superar las limitaciones de los procesos manuales actuales en ambas áreas. Para la gestión de evidencias, se abordará la dificultad de acceso, organización y control, lo que se traducirá en:

- Reducción significativa del tiempo de clasificación manual (hasta 70% de ahorro).
- Eliminación de la duplicidad de archivos (abordando 35% de discrepancias).
- Garantía del cumplimiento continuo de los estándares definidos por SINAES.
- Centralización efectiva de más de 500 documentos anuales con metadatos estructurados.
- Mejora de la seguridad y trazabilidad de la información sensible institucional.

Para el módulo de tiempos de jornada, el sistema representa una herramienta clave para la planificación eficiente de recursos humanos y académicos de la universidad, permitiendo tomar mejores decisiones sobre asignaciones docentes, ejecución de proyectos y administración del presupuesto anual de tiempos de jornada. La combinación de ambos módulos bajo el paraguas de "Gestión de Calidad" optimizará la toma de decisiones estratégicas y operativas en la UNA.

6. Objetivos

Los objetivos definen los propósitos del trabajo y los límites de lo que se desea alcanzar con el desarrollo del proyecto. Se estructuran en un objetivo general y varios objetivos específicos, redactados en infinitivo y con criterios de verificación.

6.1. Objetivo general

Desarrollar un sistema web modular que automatice la gestión de evidencias SINAES y la administración de tiempos de jornada universitaria, mejorando la trazabilidad, la transparencia y la eficiencia operativa de la UNA mediante entregas incrementales alineadas al cronograma de sprints.

6.2. Objetivos específicos

Módulo Evidencias SINAES

1. Diseñar la interfaz jerárquica (dimensión $\rightarrow$ componente $\rightarrow$ criterio) para asociar evidencias SINAES, validada con al menos 3 casos de uso completos.
2. Implementar el cargador de evidencias con metadatos obligatorios (tipo, responsable, fecha, carrera, período) y etiquetas; registrar al menos 20 evidencias de prueba.
3. Generar reportes en PDF sobre estado de cumplimiento por carrera y facultad, incluyendo filtros por período y exportación descargable.
4. Definir y aplicar un modelo de auditoría (altas, bajas, cambios, descargas) que conserve un historial mínimo de 30 días.
5. Reducir el tiempo promedio de clasificación documental a ≤ 30 minutos por usuario mediante formularios guiados y metadatos estandarizados (prueba con 3 usuarios).

Módulo Tiempos de Jornada

6. Modelar el dominio académico de tiempos de jornada (carreras, mallas, cursos, grupos, docentes, reglas) en una base NoSQL documentada.
7. Implementar los CRUD de cargas docentes y proyectos institucionales con validaciones de reglas de negocio.
8. Incorporar el cálculo automático de repitencia y tiempos externos (fijos e inciertos), mostrando el detalle del cómputo por docente.
9. Desarrollar exportes a Excel por ciclo, carrera y sede, con comparativos históricos (al menos dos períodos).
10. Publicar un prototipo web funcional (frontend + API) con autenticación, documentación Swagger y evidencia de pruebas básicas (Smoke/Happy path).

7. Metodología

7.0.1. Fundamentación

Para el desarrollo de los sistemas que conforman el macro-proyecto “**Gestión de Calidad**”, incluyendo el módulo de **gestión de evidencias** y el módulo de **gestión de tiempos de jornada**, se adoptó la metodología ágil **Scrum**.

Esta decisión se basa en la necesidad de gestionar un proyecto modular y complejo, con requisitos que evolucionan en función de los procesos de acreditación universitaria (SINAES) y de la dinámica institucional.

Las principales razones de su selección incluyen:

- **Entregas incrementales:** se realizan entregas funcionales frecuentes a los diferentes stakeholders (Comité de Calidad SINAES, administración, docentes), permitiendo retroalimentación continua.
- **Flexibilidad y adaptabilidad:** Scrum facilita la incorporación de cambios en los criterios de acreditación y en las necesidades de los usuarios sin afectar la planificación global.
- **Alineación con el cronograma académico:** la iteración por sprints se ajusta a los semestres académicos y a los avances parciales solicitados en el curso.
- **Modularidad:** la arquitectura de los módulos permite construir bases funcionales mínimas y agregar nuevas características progresivamente.

7.0.2. Referencia al WBS

El marco metodológico se complementa con la **Estructura de Desglose del Trabajo (WBS)**, que organiza los entregables y actividades principales del proyecto en fases y módulos. El WBS fue elaborado en una herramienta formal (*WBS Schedule Pro*), conforme a la observación docente, y sirve de guía para la planificación de sprints y la asignación de responsabilidades.

La Figura 7.1 muestra la estructura jerárquica del proyecto y su correspondencia con los artefactos de Scrum:

- **Nivel 1 (Proyecto):** alcance global del macro-proyecto *Gestión de Calidad*.
- **Nivel 2 (Módulos/Fases):** Evidencias SINAES, Tiempos de Jornada, Integración/QA, Documentación/Estándares, Despliegue/Cierre.
- **Nivel 3 (Entregables):** análisis, diseño UI/UX, implementación (CRUDs, reportes), pruebas, documentación.
- **Nivel 4 (Paquetes de trabajo):** tareas planificables en sprints (vinculadas al *Sprint Backlog*).

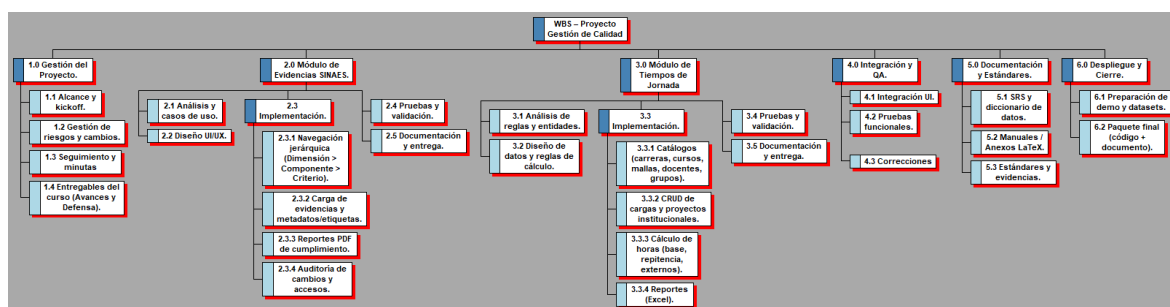


Figura 7.1: Estructura de Desglose del Trabajo (WBS) del proyecto.

Para referencia en alta resolución, el diagrama completo se incluye también en el Anexo 7: WBS (Estructura de Desglose de Trabajo).

7.0.3. Aplicación al proyecto

La adaptación de Scrum al macro-proyecto **Gestión de Calidad** se realizó con los siguientes lineamientos:

■ Roles:

- **Product Owner (PO):** Comité de Calidad de la UNA y profesores responsables, quienes priorizan y validan entregables.
- **Scrum Master:** rol asumido de forma rotativa entre los estudiantes, garantizando disciplina metodológica.
- **Equipo de Desarrollo:** integrado por los 2 estudiantes responsables de la construcción de los módulos.

■ Eventos:

- **Sprint Planning:** definición de objetivos y tareas cada dos semanas.
- **Daily Scrum:** reuniones virtuales de 15 minutos para seguimiento y bloqueos.

- **Sprint Review:** demostración de los incrementos funcionales al PO al cierre de cada sprint.
- **Sprint Retrospective:** análisis del proceso y propuestas de mejora.

■ **Artefactos:**

- **Product Backlog:** listado priorizado de requerimientos funcionales (RF-01 al RF-17).
- **Sprint Backlog:** conjunto de tareas planificadas en cada sprint.
- **Incremento:** entregables funcionales validados al cierre de la iteración.

■ **Ajustes específicos:**

- **Duración de sprint:** 2 semanas, alineadas con los avances del curso.
- **Entregas formales:** vinculadas a los Avances 1, 2, 3 y a la Defensa Final.
- **Herramientas:** GitHub Projects para gestión de backlog, GitHub Actions para CI/CD y Google Drive para almacenamiento y evidencias.

De esta forma, Scrum se implementa como una metodología realista y adaptable, con foco en entregas funcionales que aseguran el cumplimiento de los criterios académicos y de acreditación.

8. Estándares

Con el fin de garantizar la calidad, mantenibilidad, seguridad y coherencia del macro-proyecto “Gestión de Calidad” (módulo de Evidencias SINAES y módulo de Tiempos de Jornada), se definen los siguientes estándares técnicos y metodológicos.

8.1. Estándares de Documentación

- Uso de archivos **README.md** en cada repositorio con instrucciones claras de instalación, despliegue y contribución.
- Convención de **Conventional Commits** para los mensajes de confirmación en Git, asegurando trazabilidad de cambios.
- Documentación de APIs mediante **Swagger/OpenAPI 3.0**, facilitando la comprensión y consumo de los servicios.
- Manuales de usuario y anexos técnicos en formato $\text{\LaTeX}$, siguiendo la estructura institucional.

8.2. Estándares de Programación

- **Frontend:** uso de TypeScript con ESLint (Airbnb Style Guide) y Prettier para asegurar consistencia en estilo de código.
- **Backend:** aplicación de arquitectura hexagonal y principios de Domain-Driven Design (DDD) en el módulo de Tiempos de Jornada.
- **Control de versiones:** adopción de **Git Flow** para la gestión de ramas (main, develop, feature, release, hotfix).
- **Pruebas unitarias:** Jest como framework estándar para validación automática de componentes y servicios.

8.3. Estándares de Base de Datos

- **MongoDB** como base de datos NoSQL para el módulo de Tiempos de Jornada, seleccionada por su flexibilidad y escalabilidad.
- Uso de **Prisma ORM** como estándar de interacción entre backend y base de datos, asegurando tipado seguro y reducción de errores.
- **Google Drive API** como repositorio oficial de evidencias del módulo SINAES, con metadatos estandarizados para cada documento.
- Respallos periódicos automatizados, siguiendo políticas institucionales de integridad y disponibilidad.

8.4. Estándares de Interfaz

- **Shadcn UI** como librería de componentes estándar en el frontend, garantizando consistencia visual y accesibilidad.
- Cumplimiento de las pautas **WCAG 2.1 nivel AA**, asegurando accesibilidad para todos los usuarios.
- Diseño **responsive-first**, validado en dispositivos móviles, tabletas y escritorios.
- Uso de patrones de navegación jerárquica para facilitar el acceso a dimensiones, componentes y criterios en el módulo SINAES.

9. Costos y Factibilidad Económica

9.1. 3.3.3. Factibilidad Económica

Dado que se trata de un proyecto académico y sin fines de lucro, se opta por una estrategia de desarrollo de bajo costo. Se emplean herramientas y plataformas de uso libre o con planes académicos gratuitos, como GitHub para el control de versiones, MongoDB en su modalidad gratuita, y Google Drive para el almacenamiento de evidencias. El desarrollo se realiza de forma local en los equipos personales de los integrantes, distribuyendo las tareas según los módulos definidos en el WBS. Esta estrategia permite garantizar la continuidad del proyecto sin generar gastos adicionales a la institución.

9.2. Costos de desarrollo

El desarrollo se ejecuta dentro del marco del curso *Ingeniería en Sistemas II* de la Universidad Nacional, por lo que no existen costos asociados a contratación o licencias de software. Todas las herramientas utilizadas son gratuitas y de libre acceso, y el despliegue de las pruebas se realiza en entornos locales. De requerirse demostraciones institucionales, estas se efectuarán mediante el uso de cuentas académicas o servidores internos de la UNA.

9.3. Costos de capital humano

Las actividades técnicas están a cargo del equipo de desarrollo estudiantil, compuesto por:

- **Francisco Mora Cabezas:** Desarrollo del backend del módulo *Tiempos de Jornada*, diseño y definición del esquema correspondiente, integración visual y elaboración de la documentación general del proyecto.

- **Ángel Segura Méndez:** Desarrollo del backend del módulo *Evidencias SI-NAES*, modelado de su base de datos y elaboración de documentación técnica específica del módulo.

Cada integrante es responsable del ciclo completo de su módulo, incluyendo la lógica del backend, pruebas unitarias y coordinación para la integración futura entre ambos sistemas. No se contemplan remuneraciones, dado que el trabajo forma parte del proceso académico dentro del curso. La participación docente se limita a la supervisión, seguimiento metodológico y validación de los avances.

9.4. Cargos administrativos

El sistema está diseñado para operar completamente en línea o en entornos de prueba locales, eliminando la necesidad de documentación física. Por ello, los costos administrativos son prácticamente nulos. Se prevé únicamente el uso de materiales de apoyo o presentaciones digitales dentro del entorno académico.

9.5. Costos totales del proyecto

El proyecto puede desarrollarse sin generar costos monetarios directos. No obstante, se estima un gasto potencial máximo de €25 000 a €30 000 (aproximadamente 40–50 USD) en caso de requerir:

- Compra de un dominio institucional para futuras pruebas públicas.
- Certificados SSL u otros servicios asociados al despliegue de pruebas.

Estos costos son opcionales y no forman parte de la presente etapa del proyecto.

9.6. Resumen consolidado de costos

A continuación se presenta una tabla consolidada que resume todos los costos asociados al desarrollo e implementación del Sistema de Gestión Integral Universitario:

Categoría de Costo	Monto (C)	Observaciones
I. Costos de Desarrollo		
Herramientas y software	0.00	Uso de tecnologías open source
Plataformas (GitHub, MongoDB Free, Vercel)	0.00	Planes académicos gratuitos
IDE y entornos de desarrollo	0.00	VS Code, Node.js (gratuitos)
Subtotal Desarrollo	0.00	
II. Costos de Capital Humano		
Desarrollo técnico (576 horas)	0.00	Proyecto académico
Supervisión docente	0.00	Dentro del curso ISII
Reuniones y coordinación	0.00	Herramientas virtuales gratuitas
Subtotal Capital Humano	0.00	
III. Costos Administrativos		
Documentación física	0.00	Todo en formato digital
Comunicaciones	0.00	Google Meet, Discord, WhatsApp
Almacenamiento (Google Drive)	0.00	Cuentas académicas
Subtotal Administrativo	0.00	
IV. Costos Opcionales (Futuros)		
Dominio institucional (.cr)	10,000 - 15,000	Si se requiere despliegue público
Certificado SSL	5,000 - 10,000	Si se requiere HTTPS personalizado
Hosting demo (opcional)	0 - 10,000	Alternativa: usar Vercel gratuito
Subtotal Opcional	15,000 - 35,000	
TOTAL GARANTIZADO	C0.00	Costo real del proyecto
TOTAL MÁXIMO ESTIMADO	C35,000	Solo si se requieren servicios adicionales

Tabla 9.1: Consolidado General de Costos del Proyecto

Nota importante sobre costo-beneficio:

Aunque el costo directo del proyecto es de C0.00, el valor generado es significativo. La Universidad Nacional actualmente invierte aproximadamente C58 millones anuales en gestión manual de evidencias SINAES (según estimaciones institucionales de horas-persona dedicadas a búsqueda, organización y auditoría de documentos).

Adicionalmente, el módulo de Tiempos de Jornada optimiza un proceso que actualmente consume entre 40-60 horas por ciclo académico en cálculos manuales, revisiones y correcciones de asignaciones docentes. La automatización de este proceso representa

un ahorro estimado de ₡2.4 millones anuales en productividad administrativa.

Por lo tanto, la relación costo-beneficio es extraordinariamente favorable: **inversión de ₡0.00 con retorno anual estimado de ₡60+ millones en ahorro de tiempo y optimización de recursos humanos**.

9.7. Conclusiones de la factibilidad económica

El proyecto es económicamente viable y presenta una relación costo-beneficio excepcional. Aprovecha herramientas gratuitas de código abierto, no requiere inversión inicial y mantiene un costo de mantenimiento prácticamente nulo. Al estar alineado con los principios de software libre y educación abierta, contribuye al fortalecimiento institucional sin demandar recursos financieros adicionales.

El único costo potencial (₡15,000-₡35,000) surgiría únicamente si la institución decidiera en el futuro implementar un despliegue público con dominio personalizado, certificado SSL profesional y servicios premium de hosting. Sin embargo, este gasto sería completamente opcional, ya que el sistema puede operar indefinidamente con las herramientas gratuitas actuales.

En términos de costo-beneficio, la relación es altamente favorable: el impacto esperado —automatización, transparencia, eficiencia administrativa y ahorro de ₡60+ millones anuales— se logra con una inversión real de cero colones. +++

10. Desarrollo

10.1. Introducción

Este capítulo describe el proceso de desarrollo del **Sistema de Gestión Integral Universitario**, el cual contempla dos módulos principales: Gestión de Evidencias SI-NAES y Gestión de Tiempos de Jornada. El desarrollo se realiza de forma iterativa siguiendo la metodología Scrum, con entregas incrementales en cada sprint.

10.2. Gestión de Reuniones y Documentación

10.2.1. Formato de Minutas de Reunión

Durante el desarrollo del proyecto, se implementó un riguroso proceso de documentación de reuniones mediante minutas estructuradas, siguiendo el formato establecido en Anexo 6: Formato de minutas de reunión. Estas minutas constituyeron una herramienta fundamental para:

- Documentar decisiones técnicas y de negocio tomadas durante el desarrollo.
- Registrar acuerdos con los diferentes actores del proyecto.
- Establecer compromisos y fechas de entrega para cada sprint.
- Identificar problemas potenciales de manera temprana.
- Mantener la trazabilidad de los cambios solicitados.

Cada minuta siguió una estructura estandarizada que incluye:

1. Información general (fecha, hora, lugar, participantes).
2. Agenda de la reunión.
3. Temas discutidos.

4. Acuerdos alcanzados.
5. Compromisos adquiridos.
6. Próximos pasos.
7. Firmas de los participantes.

Este formato permitió mantener una comunicación efectiva entre el equipo de desarrollo y los usuarios finales del sistema (representantes de la UNA), asegurando que todas las partes interesadas estuvieran alineadas con los objetivos del proyecto.

10.2.2. Gestión de Cambios

Un aspecto crítico del desarrollo fue la implementación de un proceso formal de gestión de cambios, documentado a través de un formulario específico (ver Anexo 12: Formulario gestión de cambios). Este proceso permitió manejar de manera efectiva las modificaciones solicitadas durante el desarrollo, garantizando que:

- Los cambios fueran evaluados en términos de impacto en alcance, tiempo y recursos.
- Se mantuviera la integridad del sistema a lo largo del desarrollo.
- Se priorizaran los cambios según su importancia y urgencia.
- Se documentaran adecuadamente las modificaciones realizadas.
- Se mantuviera la trazabilidad entre los requisitos originales y los cambios implementados.

Actores Clave en la Gestión de Cambios

Para asegurar un proceso de gestión de cambios efectivo y alineado con los objetivos del proyecto, se identificaron los siguientes actores clave cuya participación fue fundamental:

- **Estudiantes (Equipo de Desarrollo):**
 - Francisco Mora Cabezas
 - Ángel Segura Méndez
- **Profesor:** Saray Castro Mora
- **Cliente (Dueño del Producto):** Josías Chávez Murillo

Un principio fundamental establecido fue que para la aprobación de cualquier cambio se requería, como mínimo, el consenso entre los estudiantes y el profesor. Este requisito garantizó que todos los cambios fueran debidamente evaluados desde perspectivas académicas y técnicas antes de su implementación.

El proceso de gestión de cambios implementado constó de las siguientes etapas:

1. **Solicitud de cambio:** Documentación formal de la modificación requerida, que podía ser iniciada por cualquiera de los actores clave.
2. **Análisis de impacto:** Evaluación técnica y de recursos necesarios, realizada por los estudiantes con supervisión del profesor.
3. **Aprobación:** Decisión basada en el análisis previo, requiriendo como mínimo el acuerdo entre estudiantes y profesor, e idealmente incluyendo al cliente.
4. **Implementación:** Desarrollo e integración del cambio por parte del equipo de estudiantes.
5. **Verificación:** Pruebas para asegurar que el cambio cumple los requisitos, con participación de todos los actores relevantes.
6. **Documentación:** Actualización de la documentación del sistema y registro formal del cambio implementado.

Esta metodología de gestión de cambios fue particularmente valiosa considerando la naturaleza iterativa del desarrollo, permitiendo incorporar mejoras y ajustes sin comprometer la calidad del producto final ni los plazos establecidos en el cronograma.

10.3. Arquitectura Técnica

El sistema se está desarrollando siguiendo una arquitectura moderna de tres capas, aplicando las mejores prácticas de la industria del software.

10.3.1. Stack Tecnológico

La selección del stack tecnológico se realizó en base a criterios de escalabilidad, mantenibilidad, compatibilidad con tecnologías actuales y disponibilidad de documentación:

- **Frontend:** Next.js 15.1.0 con React 19.0.0, utilizando el patrón App Router para enrutamiento del lado del servidor.
- **Backend:** NestJS 10.4.19, framework de Node.js que implementa arquitectura hexagonal y principios SOLID.
- **Base de Datos:** MongoDB con Prisma ORM 6.10.1, aprovechando la flexibilidad de bases de datos NoSQL para esquemas evolutivos.
- **Autenticación:** JWT (JSON Web Tokens) con Passport.js, incluyendo integración con Google OAuth2.
- **UI/UX:** shadcn/ui + Tailwind CSS 3.4.16 para interfaces responsivas y accesibles.
- **Gestión de Estado:** Zustand 4.5.4 para state management del lado del cliente.

- **Validación:** Zod 3.24.2 + class-validator para validación de datos tanto en frontend como backend.
- **Documentación API:** Swagger/OpenAPI automático integrado en NestJS.

10.3.2. Arquitectura Monorepo

El proyecto se organiza como un monorepo gestionado por pnpm workspaces y Turbo, lo que permite:

- Compartir código común entre frontend y backend (tipos TypeScript, validaciones).
- Gestionar dependencias de manera centralizada.
- Ejecutar builds incrementales y paralelos.
- Mantener consistencia en estándares de código mediante ESLint compartido.

Estructura del monorepo:

```
Gestion_Calidad/
  apps/
    backend/      # API NestJS
    frontend/     # App Next.js
    docs/         # Documentación Docusaurus
  packages/
    database/     # Prisma schemas compartidos
    ui/           # Componentes React reutilizables
    eslint-config/ # Configuraciones de linting
    typescript-config/ # tsconfig base
```

10.4. Módulo de Evidencias SINAES

10.4.1. Descripción y Cliente

El módulo de Gestión de Evidencias SINAES tiene como cliente principal al Lic. Josías Chávez Murillo. Este módulo busca centralizar y organizar toda la documentación probatoria requerida por el Sistema Nacional de Acreditación de la Educación Superior (SINAES).

10.4.2. Componentes Principales

El módulo contempla las siguientes entidades y funcionalidades:

- **Gestión de Dimensiones SINAES:** Administración de las dimensiones y componentes definidos por SINAES.

- **Tipos de Documentos:** Categorización de documentos según estándares institucionales.
- **Carga de Evidencias:** Subida y almacenamiento de documentos probatorios en Google Drive.
- **Consulta de Evidencias:** Búsqueda avanzada de documentos por dimensión, componente, criterio y año.
- **Definición de Criterios:** Establecimiento de criterios de cumplimiento para cada componente SINAES.
- **Auditoría:** Registro de cambios y modificaciones a documentos.

10.4.3. Estructura de Backend

El backend del módulo SINAES incluye los siguientes módulos NestJS:

- **DimensionsModule:** CRUD de dimensiones SINAES.
- **ComponentsModule:** Gestión de componentes por dimensión.
- **CriteriaModule:** Definición y administración de criterios de evaluación.
- **EvidencesModule:** Carga, almacenamiento y consulta de evidencias documentales.
- **DocumentTypesModule:** Categorización de tipos de documentos.
- **GoogleDriveModule:** Integración con API de Google Drive para almacenamiento.

10.4.4. Interfaz de Usuario

La interfaz del módulo SINAES proporciona vistas para:

- Dashboard con resumen de evidencias por dimensión.
- Formulario de carga de documentos con validación.
- Buscador avanzado con filtros múltiples.
- Vista de árbol jerárquico (Dimensión > Componente > Criterio > Evidencias).
- Gestión de permisos por rol (administrador, coordinador, docente).

10.5. Módulo de Tiempos de Jornada

10.5.1. Descripción y Cliente

El módulo de Gestión de Tiempos de Jornada tiene como cliente principal al Lic. Erick Madrigal. Este módulo automatiza el cálculo y distribución de las cargas académicas docentes según los tiempos de jornada establecidos por la institución.

10.5.2. Componentes Principales

El módulo contempla las siguientes funcionalidades:

- **Configuración de Rangos:** Definición de rangos de horas para cada tipo de jornada (1/4, 1/2, 3/4, tiempo completo).
- **Presupuesto Anual:** Asignación del presupuesto total de tiempos de jornada por año académico.
- **Distribución por Campus:** Asignación de tiempos de jornada a diferentes sedes universitarias.
- **Asignaciones Docentes:** Registro de cargas académicas individuales (cursos, proyectos, tareas administrativas).
- **Proyectos Institucionales:** Gestión de proyectos que requieren tiempo de jornada docente.
- **Proveedores Externos:** Administración de convenios que aportan tiempos adicionales.

10.5.3. Modelo de Datos

El modelo de datos incluye seis entidades principales en MongoDB:

1. **JourneyTimeCalculationConfig:** Almacena los rangos de horas y valores para cada tipo de jornada (1/4, 1/2, 3/4, Tiempo Completo).
2. **AnnualJourneyTimeAllocation:** Gestiona el presupuesto anual de tiempos de jornada por sede.
3. **CampusJourneyTimeAllocation:** Distribuye los tiempos entre sedes específicas, calculando automáticamente tiempos disponibles.
4. **ProfessorAssignment:** Registra asignaciones individuales de profesores a cursos, proyectos o tareas administrativas.
5. **InstitutionalProject:** Modela proyectos institucionales que consumen tiempos de jornada.
6. **ExternalProvider:** Gestiona proveedores externos (convenios) que aportan tiempos adicionales.

El modelo completo se encuentra documentado en el Anexo 15: Diccionario de Datos — Tiempos de Jornada y Anexo 9: Modelo No Relacional.

10.5.4. Estructura de Backend

El backend del módulo de Tiempos de Jornada contempla los siguientes módulos NestJS:

- **JourneyTimeConfigsModule:** Gestión de configuraciones de rangos de horas para cada tipo de jornada.
- **AnnualJourneyTimeAllocationsModule:** Administración del presupuesto anual de tiempos de jornada.
- **CampusJourneyTimeAllocationsModule:** Distribución de tiempos entre campus con cálculo automático de disponibilidad.
- **ProfessorAssignmentsModule:** Registro de asignaciones docentes individuales.
- **ExternalProvidersModule:** Gestión de proveedores externos y convenios.
- **InstitutionalProjectsModule:** Administración de proyectos institucionales.

Lógica de Cálculo

El sistema implementa cálculo automático de tiempos de jornada según las horas trabajadas. La disponibilidad de tiempos en cada campus se calcula mediante la fórmula:

$$\text{disponible} = \text{asignado} + \text{adicional} - \text{consumido}$$

Esto permite mantener control automático de la disponibilidad de recursos en cada campus.

10.5.5. Interfaz de Usuario

La interfaz del módulo de Tiempos de Jornada incluye:

- Dashboard con resumen de distribución de tiempos por campus.
- Configuración de rangos de jornada (1/4, 1/2, 3/4, tiempo completo).
- Calculadora interactiva de tipos de jornada según horas trabajadas.
- Gestión de asignaciones docentes individuales.
- Registro de proyectos institucionales y proveedores externos.
- Reportes de consumo y disponibilidad por campus y ciclo académico.

10.6. Integración entre Módulos

Ambos módulos comparten infraestructura común:

- **Autenticación:** Sistema unificado con JWT y Google OAuth2.
- **Base de Datos:** MongoDB compartida con esquemas separados por módulo.
- **Gestión de Roles:** Permisos diferenciados (administrador, coordinador, docente).
- **Auditoría:** Registro centralizado de cambios para ambos módulos.
- **API Gateway:** Punto único de entrada para servicios backend.

10.7. Estado Actual del Desarrollo

El desarrollo se encuentra en progreso continuo, con entregas incrementales en cada sprint. Las funcionalidades base de ambos módulos están implementadas, y se continúa trabajando en:

- Validaciones avanzadas en formularios.
- Mejoras en búsquedas y filtros.
- Reportes de cumplimiento y auditoría.
- Optimización de rendimiento.
- Documentación técnica y de usuario.

10.8. Implementación y Despliegue

10.8.1. Plan de Implementación del Sistema

La implementación del **Sistema de Gestión Integral Universitario** se planificó en función de los sprints definidos en el cronograma del proyecto, siguiendo un enfoque incremental que permitió integrar los módulos de Evidencias SINAES y Tiempos de Jornada de manera progresiva sobre una infraestructura común. El plan contempla las siguientes etapas:

1. **Preparación del entorno:** Configuración del monorepo (pnpm + Turbo), provisión de bases de datos MongoDB, generación de credenciales OAuth2 de Google Drive y configuración de variables de entorno por ambiente.
2. **Construcción incremental:** Desarrollo y entrega por sprints, integrando primero la base de autenticación y seguridad, luego el módulo de Evidencias SINAES y finalmente el módulo de Tiempos de Jornada.
3. **Integración continua:** Cada cambio se valida con linters (ESLint), formateo (Prettier), verificación de tipos (TypeScript) y pruebas automatizadas (Jest) antes de fusionarse a la rama principal.
4. **Despliegue a ambientes:** Promoción controlada desde el ambiente de desarrollo local hacia el ambiente de pruebas (QA) y, posteriormente, al ambiente de producción alojado en el servidor Hetzner.
5. **Verificación post-despliegue:** Revisión de bitácoras de los contenedores, pruebas de humo sobre los endpoints críticos y validación funcional de los flujos principales.

10.8.2. Estrategia de Despliegue y Ambientes Definidos

El sistema se despliega bajo una arquitectura contenedorizada con Docker y orquestada mediante `docker-compose`. Se utiliza Traefik 2.11 como proxy reverso, con resolución automática de certificados TLS mediante Let's Encrypt (HTTP-01 challenge) y cabeceras de seguridad gestionadas por archivos dinámicos de configuración.

Ambiente de Desarrollo

- **Ubicación:** Estaciones de trabajo locales del equipo de desarrollo.
- **Ejecución:** `pnpm run dev` levanta backend (NestJS) y frontend (Next.js) en modo hot-reload, junto con el servidor de documentación Docusaurus.
- **Base de datos:** Instancia MongoDB local (o cluster de desarrollo) gestionada con Prisma.
- **Propósito:** Implementación y prueba de funcionalidades, depuración y validación temprana del comportamiento.

Ambiente de Pruebas (QA)

- **Ubicación:** Construcción Docker equivalente a producción, ejecutada en una rama o instancia separada.
- **Datos:** Conjunto controlado de datos de prueba que replica escenarios reales sin comprometer información sensible.
- **Propósito:** Validación funcional, pruebas de integración, pruebas de regresión y revisión por parte del cliente antes de cada liberación.

Ambiente de Producción

- **Infraestructura:** Servidor dedicado en Hetzner, red Docker aislada `gcalidad-network` (172.30.0.0/24).
- **Servicios:** Contenedores `agr-frontend` (Next.js), `agr-backend` (NestJS) y `traefik` actuando como proxy y terminador TLS.
- **Dominios:** Frontend expuesto en `gestion-calidad.arayaroma.software` y API en `api-gestion-calidad.arayaroma.software`, con redirección automática HTTP → HTTPS.
- **Almacenamiento de archivos:** Google Drive como repositorio documental para las evidencias SINAES, mediante una cuenta de servicio con acceso restringido.

10.8.3. Procedimiento de Instalación y Configuración

El despliegue automatizado se realiza mediante el script `deploy-hetzner.sh`, que sincroniza el código fuente con el servidor remoto y reconstruye los contenedores. El procedimiento general es el siguiente:

1. **Requisitos previos:** Node.js $\geq 18.x$, pnpm $\geq 8.x$, Docker y Docker Compose instalados en el servidor.
2. **Clonado y configuración:** Obtención del repositorio y creación de los archivos `.env`, `apps/backend/.env.production` y `apps/frontend/.env.production` con las credenciales correspondientes (cadena de conexión a MongoDB, secretos JWT, credenciales Google OAuth2, dominios públicos).
3. **Sincronización:** `./deploy-hetzner.sh` all ejecuta `rsync` excluyendo `node_modules`, `.next`, `dist`, `.git` y archivos sensibles locales.
4. **Construcción de imágenes:** `docker compose build --no-cache` reconstruye las imágenes de `frontend` y `backend`.
5. **Levantamiento de servicios:** `docker compose up -d` inicia los contenedores con sus etiquetas Traefik para enrutamiento y certificados.
6. **Verificación:** Inspección de `docker compose ps` y de las bitácoras (`docker logs agr-frontend`, `docker logs agr-backend`) para confirmar el inicio correcto.
7. **Migración de esquemas:** Ejecución de `pnpm run prisma` para generar y aplicar los esquemas de Prisma sobre la base de datos.

10.8.4. Plan de RespalDOS y Recuperación

Dado que el sistema gestiona información crítica para la acreditación SINAES y la planificación académica, se define una estrategia de respaldos en dos planos:

- **Base de datos (MongoDB):** RespalDOS automatizados mediante `mongodump` con periodicidad diaria (incrementales) y semanal (completos), almacenados de forma cifrada y conservados por al menos 30 días. Las restauraciones se prueban periódicamente sobre un ambiente aislado para validar la integridad de los volcados.
- **Documentos probatorios (Google Drive):** La propia plataforma de Google Drive provee versionado y papelera con periodo de retención. Adicionalmente, se documenta una jerarquía de carpetas controlada por el módulo `GoogleDriveFolders`, lo que permite reconstruir la estructura ante eliminaciones accidentales.
- **Configuración e infraestructura:** El `docker-compose.yml`, los archivos de Traefik (`headers.yml`, `redirect.yml`, `tls.yml`) y los certificados de Let's Encrypt (volumen `traefik_certs`) se versionan o respaldan para permitir la reconstrucción rápida del entorno.
- **Procedimiento de recuperación:** En caso de incidente se restaura primero la infraestructura base (servidor, red Docker, Traefik), luego la base de datos a partir del último respaldo válido y, finalmente, se redespliega la aplicación mediante `deploy-hetzner.sh`.

10.8.5. Capacitación al Usuario Final y Entrega de Manuales

La estrategia de capacitación se concibió en dos frentes, alineados con los perfiles de usuario del sistema:

- **Capacitación funcional:** Sesiones dirigidas a los usuarios clave (Lic. Josías Chávez Murillo para el módulo SINAES y Lic. Erick Madrigal para el módulo de Tiempos de Jornada), cubriendo los flujos de carga de evidencias, configuración de rangos de jornada, registro de asignaciones docentes y consulta de reportes.
- **Material entregable:** Documentación funcional y técnica generada con Docusaurus (`apps/docs`), accesible vía `pnpm --filter @una-gc/docs start` en `http://localhost:5000`, complementada con la documentación API auto-generada por Swagger/OpenAPI desde el backend NestJS.
- **Manuales:** Manual de usuario por módulo, manual de administrador (gestión de roles y permisos) y manual técnico de despliegue, todos versionados dentro del propio monorepo.

10.9. Aceptación por Parte del Usuario

10.9.1. Plan de Pruebas de Aceptación del Usuario (UAT)

El plan de UAT se diseñó para validar, en condiciones reales de uso, que el sistema cumple con los requerimientos definidos en el SRS (Anexo 8). Contempla:

- **Alcance:** Validación de los flujos críticos de ambos módulos: autenticación con Google OAuth2, carga y búsqueda de evidencias SINAES, configuración de rangos de jornada, registro de asignaciones docentes y generación de reportes.
- **Criterios de aceptación:** Cumplimiento de los requerimientos funcionales del SRS, ausencia de defectos bloqueantes, tiempos de respuesta razonables en operaciones cotidianas y consistencia entre la información mostrada en la interfaz y la almacenada en MongoDB / Google Drive.
- **Roles participantes:** Cliente / Dueño del Producto (Lic. Josías Chávez Murillo y Lic. Erick Madrigal), Profesor responsable (Saray Castro Mora) y Equipo de Desarrollo.
- **Casos de prueba:** Conjunto derivado de las historias de usuario, ejecutados sobre el ambiente de QA con datos de prueba representativos.

10.9.2. Ejecución y Evidencias de las Pruebas

La ejecución de las pruebas de aceptación se realizó en sesiones conjuntas con los clientes sobre el ambiente desplegado. Las evidencias técnicas se recopilan en el Anexo 13: Desarrollo, que contiene capturas de las vistas funcionales del sistema. Adicionalmente, el repositorio cuenta con una suite de pruebas automatizadas (Jest) que respalda

la validación funcional ejecutada por el usuario, con reportes de cobertura disponibles vía `pnpm --filter @una-gc/backend test:cov`.

10.9.3. Bitácora de Hallazgos y Correcciones Aplicadas

Durante el desarrollo se mantuvo un proceso continuo de auditoría técnica del módulo SINAES, formalizado en el documento `SINAES_AUDIT.md` dentro del repositorio. Esta bitácora documenta:

- Inconsistencias críticas (duplicaciones de componentes, código stub en producción).
- Errores de manejo de errores y casos borde (validaciones faltantes, `catch` silenciosos).
- Recomendaciones de remediación con su correspondiente prioridad.

Las correcciones se gestionaron a través del proceso formal de gestión de cambios descrito al inicio del capítulo, dejando registro en commits del repositorio y en el plan de remediación `SINAES_FIX_PLAN.md`.

10.9.4. Acta de Aceptación

El acta o carta formal de aceptación firmada por el usuario se incorpora como anexo del documento una vez completada la ejecución de la UAT en presencia del cliente y del profesor responsable (ver Anexo 12: Carpeta de firmas).

10.10. Desafíos

10.10.1. Desafíos Técnicos Enfrentados

Durante el desarrollo se enfrentaron varios desafíos relevantes:

- **Integración con Google Drive:** Manejo de cuotas de la API, autenticación OAuth2 con cuenta de servicio, sincronización entre la jerarquía documental en Drive y la base de datos, y construcción de un mecanismo de verificación de consistencia (`DriveSyncCheckerService`).
- **Modelado de la jerarquía SINAES:** Representación en MongoDB de una estructura de 5 niveles (Dimensiones → Componentes → Criterios → Estándares/Evidencias Directas → Evidencias de Calidad) preservando integridad referencial sobre un motor NoSQL.
- **Cálculo de tiempos de jornada:** Implementación de la fórmula `disponible = asignado + adicional - consumido` con consistencia transaccional sobre múltiples entidades (configuración, presupuesto anual, distribución por campus, asignaciones docentes).

- **Monorepo multi-app:** Coordinación de dependencias compartidas, builds incrementales y configuración unificada de ESLint/TypeScript entre backend, frontend, paquetes UI y documentación.
- **Despliegue contenedorizado:** Configuración de Traefik con resolución automática de certificados TLS, cabeceras de seguridad y enrutamiento dual HTTP/HTTPS para frontend y backend bajo dominios distintos.

10.10.2. Riesgos Materializados y Acciones de Mitigación

- **Inconsistencias en código de producción:** Se detectaron componentes de depuración (`scroll-debug`, `config-debug`) y duplicaciones de archivos críticos en el módulo SINAES. **Mitigación:** formalización de la auditoría en `SINAES_AUDIT.md`, definición del plan de remediación en `SINAES_FIX_PLAN.md` y aislamiento de utilidades de depuración detrás de `NODE_ENV === 'development'`.
- **Funcionalidad parcial en sincronización con Drive:** El servicio `DriveSyncCheckerService` operaba como *stub* y retornaba un estado optimista. **Mitigación:** priorización de la implementación real y bloqueo de la operación en producción mientras se cierra el hallazgo.
- **Dependencia de servicios externos (Google Drive, Let's Encrypt):** Disponibilidad sujeta a terceros. **Mitigación:** manejo de reintentos con backoff exponencial, monitoreo de cuotas y persistencia de los certificados en un volumen Docker (`traefik_certs`).

10.10.3. Cambios de Alcance y su Impacto

Los cambios solicitados durante el desarrollo se canalizaron a través del Formulario de Gestión de Cambios (Anexo 11), evaluando en cada caso el impacto en alcance, cronograma y recursos. La metodología iterativa (Scrum) permitió absorber estos cambios en sprints siguientes sin comprometer la fecha de entrega general del proyecto.

10.10.4. Lecciones Aprendidas

- Mantener una **auditoría técnica continua** (no solo al cierre) detecta inconsistencias antes de que se conviertan en deuda crítica.
- Las **verificaciones automatizadas** (linters, type-check, pruebas unitarias, cobertura) son indispensables en un monorepo donde un cambio puede impactar varias aplicaciones.
- Documentar la **infraestructura como código** (`docker-compose`, scripts de despliegue) acelera la recuperación ante incidentes y la incorporación de nuevos integrantes al equipo.
- La **comunicación frecuente con los clientes** (Lic. Chávez y Lic. Madrigal) permitió ajustar prioridades de forma temprana y evitar reprocesos.

- Disociar lógica de almacenamiento (Google Drive) de lógica de negocio (MongoDB/Prisma) facilitó las pruebas y la evolución independiente de cada componente.

10.11. Detalles Técnicos

10.11.1. Arquitectura del Sistema

El sistema sigue una arquitectura de tres capas desplegada como microservicios contenedorizados detrás de un proxy reverso:

- **Capa de presentación:** Aplicación Next.js (App Router) que consume la API REST mediante un cliente HTTP centralizado, con estado global gestionado por Zustand y caché de servidor con React Query.
- **Capa de aplicación:** API NestJS modular, con módulos independientes por dominio (más de 50 módulos en `apps/backend/src/modules/`, agrupados por las áreas SINAES, Tiempos de Jornada, autenticación, parametrización y soporte).
- **Capa de datos:** MongoDB accedido vía Prisma ORM con esquemas compartidos en el paquete `@una-gc/database`, complementado por Google Drive como almacén de objetos documentales.

El diagrama detallado de componentes está disponible en el Anexo 18: Diseño, junto con los diagramas específicos de cada módulo (Anexo 19 y Anexo 20).

10.11.2. Tecnologías, Lenguajes y Frameworks Utilizados

- **Lenguajes:** TypeScript (frontend, backend y paquetes compartidos), JavaScript (scripts de soporte), SQL/MongoDB Query Language para consultas.
- **Frontend:** Next.js 15.1.0, React 19.0.0, Tailwind CSS 3.4.16, shadcn/ui, Zustand 4.5.4, React Query.
- **Backend:** NestJS 10.4.19, Prisma ORM 6.10.1, Passport.js, JWT, Zod 3.24.2, class-validator.
- **Persistencia:** MongoDB.
- **Integraciones externas:** Google Drive API (`googleapis`), Google OAuth2.
- **Tooling:** pnpm workspaces, Turbo, ESLint, Prettier, Husky (hooks de git), Jest (pruebas unitarias y de integración), Docusaurus (documentación).
- **Infraestructura:** Docker, Docker Compose, Traefik 2.11, Let's Encrypt, servidor Hetzner.

10.11.3. Repositorio del Código Fuente y Control de Versiones

El código fuente se gestiona en un repositorio Git con flujo de trabajo basado en ramas por funcionalidad y revisiones por pares antes de fusión a la rama principal. Aspectos relevantes del control de versiones:

- **Estructura:** monorepo con apps (`backend`, `frontend`, `docs`) y paquetes (`database`, `ui`, `eslint-config`, `typescript-config`).
- **Hooks pre-commit:** configurados con Husky para ejecutar linters, formateadores y verificaciones de ortografía técnica (`cspell.config.yaml`).
- **Convenciones de commits:** mensajes descriptivos en español/inglés vinculados a tareas e issues del cronograma de sprints.
- **Equipo y contribuyentes:** Juan C. Camacho Solano, Fran Mora Cabezas, Ángel Stward Segura Méndez y Esteban Javier Granados Sibaja.

10.11.4. Estándares de Codificación Aplicados

- **Linting:** configuración compartida en `@una-gc/eslint-config` para asegurar consistencia entre paquetes.
- **Tipado estricto:** TypeScript con `strict` habilitado, tipos compartidos publicados en paquetes internos.
- **Arquitectura:** principios SOLID en NestJS, separación módulo/servicio/controlador, repositorios para acceso a datos.
- **Validación:** DTOs validados con `class-validator` en backend y esquemas Zod en frontend.
- **Pruebas:** Jest con generación incremental de pruebas por fases (`generate_tests_phase*.sh`) y reportes de cobertura.
- **Estilo:** Prettier para formateo automático y reglas de nomenclatura consistentes (kebab-case en archivos, camelCase en variables, PascalCase en componentes y clases).

10.11.5. Requerimientos de Hardware y Software para la Ejecución

Servidor de producción (mínimo recomendado):

- CPU: 2 vCPU.
- Memoria RAM: 4 GB.
- Almacenamiento: 40 GB SSD para sistema, imágenes Docker y volúmenes persistentes.
- Sistema operativo: Linux con Docker Engine y Docker Compose v2.

- Conectividad: IP pública con puertos 80 y 443 abiertos, registros DNS apuntando a los dominios del frontend y del backend.

Estación de desarrollo:

- Node.js $\geq 18.x$ y pnpm $\geq 8.x$.
- Docker Desktop o equivalente para pruebas locales contenedorizadas.
- Acceso a MongoDB local o a un cluster de desarrollo.

Cliente final: navegador moderno (Chrome, Firefox, Edge o Safari en sus versiones recientes) con JavaScript habilitado.

10.11.6. Seguridad: Autenticación, Autorización y Manejo de Datos Sensibles

Autenticación

- Inicio de sesión mediante JWT firmado, gestionado con Passport.js dentro del módulo `auth` del backend.
- Integración con Google OAuth2 para el flujo de autenticación federada, requerido también para autorizar el acceso a Google Drive en nombre del usuario.
- Tokens almacenados de forma segura en el cliente y renovados según política definida en el backend.

Autorización

- Modelo de roles diferenciados (administrador, coordinador, docente) implementado en los módulos `user-roles` y `user-permissions`.
- *Guards* y decoradores específicos de NestJS protegen los endpoints sensibles (`apps/backend/src/modules/auth/guards` y `decorators`).
- Permisos verificados tanto en el backend (autoridad final) como en el frontend (para condicionar la interfaz).

Manejo de Datos Sensibles

- **Cifrado en tránsito:** TLS terminado en Traefik con certificados Let's Encrypt y redirección forzada de HTTP a HTTPS.
- **Cabeceras de seguridad:** configuradas en `traefik/headers.yml` (HSTS, anti-clickjacking, protección contra MIME-sniffing).
- **Secretos:** variables de entorno gestionadas por archivos `.env` fuera del repositorio, con plantillas controladas.
- **Credenciales de servicio:** cuenta de servicio Google con permisos mínimos sobre Drive, almacenada como archivo JSON fuera del bundle de la imagen.

- **Auditoría:** registro centralizado de cambios en módulos clave (`sinaes-document-history`, `observations`, `project-logs`) para trazabilidad de acciones sensibles.
- **Aislamiento de red:** contenedores en una red Docker dedicada (`gcalidad-network`) sin exposición directa, expuestos únicamente a través del proxy.

11. Conclusiones

El proyecto del ****Sistema de Gestión Integral Universitario****, actualmente en sus fases iniciales de planificación y definición de requerimientos, se perfila como una solución estratégica para la Universidad Nacional. Tras la etapa de análisis preliminar y la consolidación de la Especificación de Requerimientos de Software (SRS), se pueden establecer las siguientes conclusiones sobre su potencial y el camino a seguir:

- **Viabilidad y Pertinencia del Proyecto:** Se ha confirmado la viabilidad técnica y operativa del Sistema de Gestión Integral Universitario, el cual responde a necesidades críticas identificadas en la gestión de evidencias para acreditación SINAES y la administración de tiempos de jornada universitaria. Este proyecto es pertinente y fundamental para la modernización y optimización de procesos académicos y administrativos clave de la UNA.
- **Claridad en la Definición de Requerimientos:** A través de un proceso riguroso de recopilación y consolidación, se ha logrado definir una Especificación de Requerimientos de Software (SRS) completa, coherente y suficientemente detallada para guiar las próximas fases de diseño y desarrollo. Los requerimientos funcionales y no funcionales de ambos módulos han sido claros y validados con los usuarios clave.
- **Alineación con Estándares y Metodologías Modernas:** La adopción de metodologías ágiles como Scrum y el uso de estándares de desarrollo y seguridad modernos (React/Next.js, NestJS, MongoDB, Prisma, Google Drive API, JWT, AES-256) proveen una base tecnológica y de gestión robusta para el proyecto, prometiendo un desarrollo eficiente y de alta calidad.
- **Potencial de Impacto Significativo:** Se proyecta que el sistema contribuirá a la reducción de tiempos administrativos (ej. clasificación de evidencias), la eliminación de duplicidades, la mejora en la precisión de la planificación de recursos humanos y el fortalecimiento de la trazabilidad y seguridad de la información institucional. Estos beneficios son cruciales para los procesos de acreditación y la eficiencia operativa de la UNA.

- **Base para Futuras Fases:** Los requerimientos definidos y la arquitectura modular propuesta para el Sistema de Gestión Integral Universitario sientan una base sólida para su desarrollo en el curso de Ingeniería en Sistemas I y para futuras expansiones y mejoras por parte de la Universidad, consolidando una plataforma integral de gestión de la calidad.

11.1. Logros hasta la Fecha (Fase de Planificación)

En esta etapa inicial del proyecto, los principales logros se centran en la preparación y el establecimiento de una base sólida:

- **Conformación y Coordinación del Equipo de Proyecto:** El equipo de estudiantes ha demostrado capacidad de organización y colaboración efectiva, estableciendo canales de comunicación claros con la profesora y los clientes.
- **Definición Exhaustiva del Alcance y Requerimientos:** Se han identificado y documentado los requerimientos funcionales y no funcionales esenciales para ambos módulos, culminando en un SRS consolidado que servirá como guía principal para el desarrollo.
- **Planificación Detallada del Desarrollo:** Se ha establecido un cronograma de Sprints y una estrategia de desarrollo ágil que permitirá la entrega incremental de funcionalidades, optimizando el uso del tiempo disponible en el curso.
- **Análisis de Viabilidad Confirmado:** Los estudios de factibilidad técnica, operativa y económica han ratificado que el proyecto es viable con los recursos y tecnologías disponibles en el entorno universitario.

En síntesis, aunque el desarrollo del software apenas comienza, la fase de planificación ha sido exitosa en establecer una dirección clara, un conjunto de requerimientos bien definidos y una base estratégica sólida para el Sistema de Gestión Integral Universitario.

12. Recomendaciones

A partir de la planificación y el inicio del desarrollo del Sistema de Gestión Integral Universitario (que incluye los módulos de Gestión de Evidencias SINAES y Gestión de Tiempos de Jornada), y con el objetivo de asegurar su éxito a largo plazo y su evolución óptima, se presentan las siguientes recomendaciones estratégicas:

12.0.1. Enfoque Técnico y Arquitectónico

- **Priorizar el diseño modular y escalable de la arquitectura** para ambos módulos desde las fases iniciales. Esto permitirá la fácil integración de nuevas funcionalidades y futuras expansiones sin requerir rediseños mayores. Se recomienda enfocarse en:
 - Definición clara de interfaces y APIs entre componentes.
 - Uso de microservicios o arquitecturas distribuidas para el módulo de Tiempos de Jornada.
 - Estrategias de desacoplamiento para minimizar dependencias.
 - Adopción de patrones de diseño que faciliten la escalabilidad horizontal.
 - Consideración de un diseño "API-first" para futuras integraciones.
- **Establecer una estrategia robusta de respaldo y recuperación de datos** desde el inicio del proyecto. Esto es crucial para la información de evidencias (Google Drive) y los datos de tiempos de jornada (MongoDB). La estrategia debe incluir:
 - Definición de la frecuencia de respaldos (diarios, semanales).
 - Mecanismos de almacenamiento de respaldos en ubicaciones seguras y geográficamente dispersas.
 - Protocolos de cifrado para datos en reposo y en tránsito.
 - Pruebas periódicas y documentadas de restauración de datos.
 - Un plan de recuperación ante desastres (DRP) detallado.

- **Planificar la integración con la API de Google Drive** con un enfoque en la robustez y manejo de excepciones. Dado que el módulo de evidencias no tendrá un backend propio en la primera fase, es vital asegurar:
 - Manejo de límites de cuota y errores de la API.
 - Estrategias de reintento con retroceso exponencial.
 - Monitoreo de la salud de la conexión y las operaciones con Drive.
 - Auditoría de cambios y sincronización para prevenir inconsistencias.
 - Documentación exhaustiva de los flujos de interacción con Drive.
- **Investigar y planificar futuras capacidades móviles** para ambos módulos, considerando la creciente necesidad de acceso desde dispositivos. Esto podría incluir:
 - Diseño responsive para el frontend web.
 - Exploración de frameworks para desarrollo de aplicaciones híbridas.
 - Identificación de funcionalidades clave para un acceso móvil prioritario.
 - Consideración de modos offline para la captura de datos en áreas sin conectividad.
 - Análisis de las necesidades de rendimiento y experiencia de usuario móvil.

12.0.2. Estrategias Operativas y de Adopción

- **Diseñar un programa de capacitación integral** para los usuarios finales (Gestores de Calidad, Decanos, Directores de Carrera) desde las etapas tempranas del desarrollo. Este programa debe:
 - Adaptar los contenidos a los roles específicos y niveles de competencia tecnológica.
 - Incluir sesiones prácticas y materiales de apoyo (manuales, videos).
 - Fomentar la participación activa y la resolución de dudas en vivo.
 - Establecer un cronograma de capacitaciones pre-lanzamiento y post-lanzamiento.
 - Designar usuarios clave como "campeones" del sistema para soporte inicial.
- **Establecer mecanismos claros de retroalimentación de usuarios** y gestión de solicitudes de mejora continua. Esto facilitará la identificación temprana de problemas y oportunidades de optimización. Se sugiere:
 - Implementar un canal centralizado para reportar errores y sugerencias.
 - Definir un proceso para el análisis y priorización de estas solicitudes.
 - Comunicar proactivamente el estado de las mejoras implementadas a los usuarios.
 - Realizar encuestas de satisfacción y entrevistas con usuarios periódicamente.
 - Fomentar una cultura de mejora continua en la comunidad de usuarios.

- **Definir Indicadores Clave de Desempeño (KPIs)** para ambos módulos desde la fase de análisis. Estos KPIs permitirán medir el impacto del sistema en la eficiencia operativa y el cumplimiento de objetivos. Deben incluir:
 - Métricas de tiempo (ej. tiempo de clasificación de evidencia, tiempo de registro de jornada).
 - Métricas de calidad de datos (ej. porcentaje de documentos con metadatos completos).
 - Métricas de cumplimiento (ej. porcentaje de criterios SINAES con evidencias asociadas).
 - Métricas de uso del sistema y satisfacción del usuario.
 - Establecimiento de líneas base y metas realistas.
- **Establecer un comité de gobernanza del proyecto** con representación de los principales stakeholders (académicos, administrativos, TI). Este comité debería:
 - Reunirse periódicamente para supervisar el progreso y la alineación estratégica.
 - Aprobar cambios significativos en el alcance y prioridades.
 - Gestionar riesgos de alto nivel y facilitar la resolución de conflictos.
 - Promover la adopción del sistema a nivel institucional.
 - Asegurar la disponibilidad de recursos necesarios para el proyecto.

12.0.3. Crecimiento y Alcance Futuro

- **Diseñar el sistema con una visión de integración con otros sistemas universitarios** a futuro. Aunque no sea parte de la fase inicial, identificar puntos de integración potenciales (ej. sistemas de recursos humanos, gestión académica, finanzas) es crucial para:
 - Evitar silos de información y duplicidad de datos.
 - Optimizar procesos transversales en la UNA.
 - Maximizar el valor de la información institucional.
 - Sentar las bases para una visión de campus inteligente.
 - Reducir la carga administrativa al automatizar flujos de datos.
- **Explorar el potencial de análisis avanzado y predicción** para ambos módulos una vez que se acumulen suficientes datos históricos. Esto podría incluir:
 - Para evidencias SINAES: Predecir posibles deficiencias en criterios.
 - Para Tiempos de Jornada: Optimización de la asignación de recursos docentes.
 - Identificación de patrones y tendencias para la toma de decisiones estratégicas.

- Integración con herramientas de Business Intelligence.
- Desarrollo de dashboards interactivos con proyecciones.
- **Planificar la expansión de las capacidades de reportería** para satisfacer necesidades analíticas avanzadas más allá de los reportes iniciales en PDF/Excel. Las mejoras futuras podrían incluir:
 - Herramientas de reportes personalizables por el usuario final.
 - Implementación de visualizaciones de datos dinámicas.
 - Capacidades de análisis multidimensional (OLAP).
 - Integración con plataformas de datos abiertos de la UNA.
 - Generación programada y distribución automática de informes clave.

12.0.4. Gestión del Conocimiento y Documentación

- **Mantener una documentación técnica y de usuario actualizada** a lo largo de todo el ciclo de vida del proyecto. Esta debe ser una tarea continua, no solo al final, para asegurar que refleje la evolución del sistema. Incluir:
 - Un proceso formal de actualización vinculado a cada sprint y liberación.
 - Diagramas de arquitectura y flujos de trabajo detallados.
 - Documentación de decisiones técnicas clave y su justificación.
 - Ejemplos prácticos y casos de uso para los usuarios.
 - Un sistema de control de versiones para la documentación.
- **Establecer procedimientos de transferencia de conocimiento** efectivos dentro del equipo de desarrollo y hacia el personal de TI de la UNA. Esto es crucial para la sostenibilidad y mantenimiento futuro. Se debe considerar:
 - Sesiones de walkthrough de código y diseño.
 - Definir protocolos para el traspaso de responsabilidades.
 - Mantener registros detallados de configuraciones y personalizaciones.
 - Documentar soluciones a problemas recurrentes y lecciones aprendidas.
 - Establecer repositorios centralizados de conocimiento técnico.
- **Crear una base de conocimiento de soporte** para los usuarios y el equipo de TI. Esta base permitirá la resolución rápida de incidencias y reducirá la dependencia del equipo de desarrollo inicial. Debería:
 - Organizar soluciones por categorías intuitivas de problemas frecuentes.
 - Incluir capturas de pantalla y videos instructivos.
 - Proporcionar procedimientos paso a paso para la resolución de incidencias.
 - Implementar un sistema de búsqueda eficiente por palabras clave.
 - Permitir la contribución y calificación de soluciones por parte de los usuarios.

- **Documentar las lecciones aprendidas** en cada fase del proyecto, especialmente en los momentos clave (ej. al finalizar cada sprint, al cierre del proyecto). Esta práctica contribuirá al mejoramiento de futuros proyectos de desarrollo en la UNA. La documentación debe:
 - Analizar aspectos exitosos y áreas de mejora en el proceso.
 - Identificar factores críticos de éxito replicables.
 - Documentar obstáculos encontrados y estrategias efectivas de superación.
 - Evaluar la efectividad de diferentes enfoques de gestión del cambio.
 - Proporcionar recomendaciones concretas para futuros proyectos tecnológicos.

12.0.5. Sostenibilidad a Largo Plazo

Para garantizar la viabilidad y relevancia del sistema en el largo plazo, se recomienda:

- **Asegurar la asignación de un presupuesto anual dedicado** al mantenimiento y mejora continua del sistema una vez puesto en producción. Esto debe incluir recursos para actualizaciones tecnológicas, capacitación continua y desarrollo de nuevas funcionalidades.
- **Fomentar la capacitación interna del personal de TI de la UNA** en las tecnologías utilizadas (React, Next.js, TypeScript, NestJS, MongoDB, Prisma) para reducir la dependencia de proveedores externos o del equipo de desarrollo inicial, asegurando la capacidad de soporte y evolución interna.
- **Establecer alianzas estratégicas con la academia o centros de investigación** de la misma UNA para proyectos de investigación aplicada que potencien el sistema a futuro, como la integración de IA para análisis predictivos o la automatización de procesos.
- **Evaluar periódicamente la alineación del sistema** con los objetivos estratégicos de la UNA, las normativas SINAES y las mejores prácticas en gestión universitaria, asegurando su relevancia y pertinencia continua.
- **Promover la participación activa en comunidades de desarrolladores y usuarios** de las tecnologías empleadas. Esto facilitará el intercambio de conocimientos, la identificación de nuevas funcionalidades y la resolución colaborativa de desafíos técnicos.

A. Anexos

Anexo 1: Carta de presentación

Documento adjunto con la carta de presentación formal del proyecto "Sistema de Gestión Integral Universitario". (Abrir anexo)

Anexo 2: Carta de intenciones

Documento adjunto con la carta formal que expresa la intención de las partes involucradas. (Abrir anexo)

Anexo 3: Código de Ética

Código de ética a seguir por todos los miembros del proyecto. (Abrir anexo)

Anexo 4: Propuesta del proyecto

Propuesta formal del proyecto (empresa patrocinadora y alcances). (Abrir anexo)

Anexo 5: Registro de actores del proyecto

Listado y roles de todos los actores del proyecto. (Abrir anexo)

Anexo 6: Formato de minutas de reunión

Plantilla estándar para actas y acuerdos. (Abrir anexo)

Anexo 7: WBS (Estructura de Desglose de Trabajo)

Diagrama con la estructura jerárquica de entregables. (Abrir anexo)

Anexo 8: SRS

Especificación de Requerimientos de Software (funcionales y no funcionales). (Abrir anexo)

Anexo 9: Modelo No Relacional

Modelo no relacional (Prisma). (Abrir anexo)

Anexo 10: Diccionario de Datos (general)

Tablas, campos, tipos, restricciones y relaciones del sistema. (Abrir anexo)

Anexo 11: Formulario de gestión de cambios

Formulario para solicitar/analizar/aprobar cambios (alcance, cronograma, recursos). (Abrir anexo)

Anexo 12: Carpeta de firmas

Documentos originales firmados (aprobaciones, minutas, acuerdos). (Abrir anexo)

Anexo 13: Desarrollo

Evidencias de vistas y prototipos funcionales. (Abrir anexo)

Anexo 14: Diccionario de Datos — Evidencias SINAES

Estructura y metadatos del módulo de evidencias SINAES. (Abrir anexo)

Anexo 15: Diccionario de Datos — Tiempos de Jornada

Estructura y reglas de cálculo del módulo de tiempos de jornada. (Abrir anexo)

Anexo 16: Glosario

Glosario de términos y abreviaturas del proyecto. (Abrir anexo)

Anexo 17: Análisis

Documentos de análisis, supuestos y límites. (Abrir anexo)

Anexo 18: Diseño

Diagramas y artefactos de diseño (UI/arquitectura). (Abrir anexo)

Anexo 19: Diagrama Evidencias SINAES

Diagrama general del módulo de Evidencias.

Enlace: [Abrir anexo](#)

Anexo 20: Diagrama Tiempos de Jornada

Diagrama general del módulo de Tiempos de Jornada.

Enlace: [Abrir anexo](#)

Anexo 21 : Cronograma de Sprints

En la Figura A.1 se presenta el cronograma de sprints planificado para el proyecto de **Ingeniería en Sistemas II**, el cual contempla tanto los avances técnicos como la documentación asociada a los módulos de *Evidencias SINAES* y *Tiempos de Jornada*. Este cronograma sigue la estructura de entregables definidos en los sprints, permitiendo visualizar el estado (Completado, En Proceso, Pendiente) y las fechas clave de ejecución.

Versión digital: [Abrir cronograma](#)

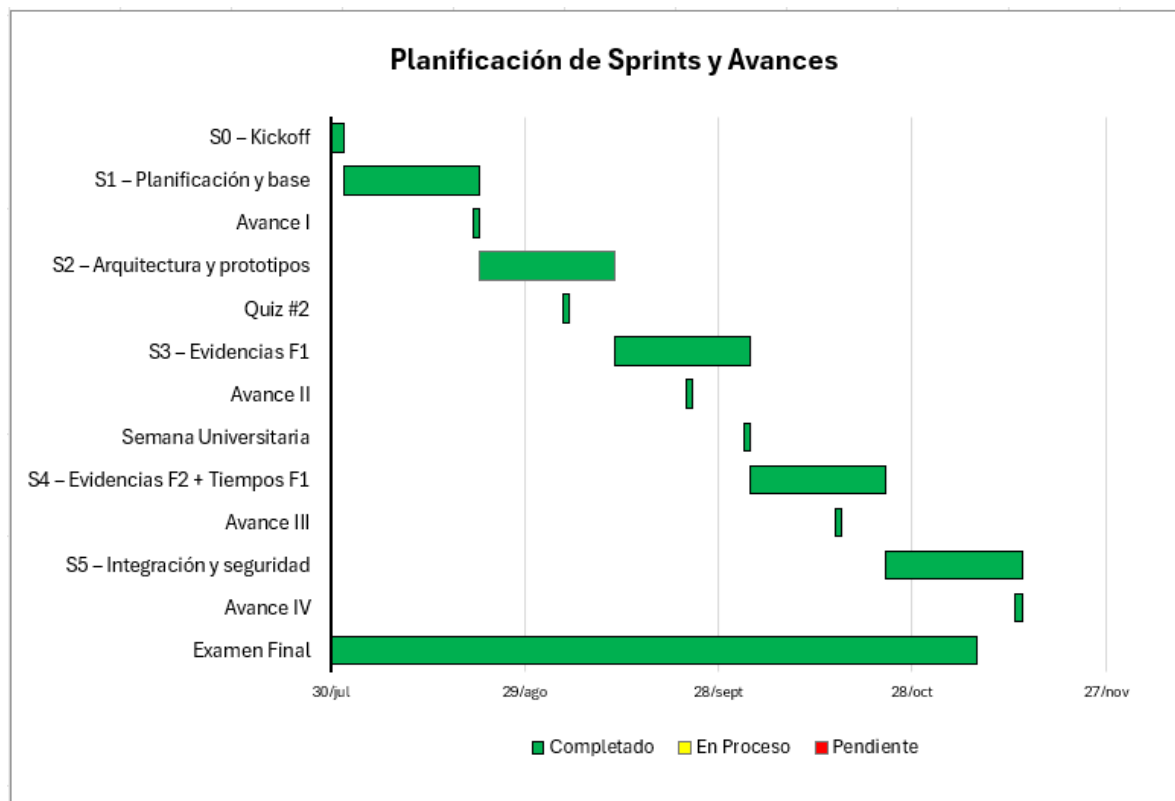


Figura A.1: Cronograma de Sprints del Proyecto Ingeniería en Sistemas II.

Referencias bibliográficas

- [Inf, 2024] (2024). Lineamientos de evaluación sinaes.
- [Reg, 2024] (2024). Reglamento de tiempos de jornada, una.
- [SIN, 2025] (2025). Sinaes - sistema nacional de acreditación. <https://www.sinaes.ac.cr>.
- [UNA, 2025] (2025). Universidad nacional de costa rica. <https://www.una.ac.cr>.
- [Chacon and Straub, 2023] Chacon, S. and Straub, B. (2023). *Pro Git*. Apress.
- [Cohn, 2017] Cohn, M. (2017). *Succeeding with Agile: Software Development Using Scrum*. Addison-Wesley Professional.
- [Contraloría General de la República de Costa Rica, 2022] Contraloría General de la República de Costa Rica (2022). Normas técnicas para la gestión y el control de las tecnologías de información.
- [Date, 2019] Date, C. J. (2019). *Database Design and Relational Theory: Normal Forms and All That Jazz*. O'Reilly Media.
- [Ministerio de Salud de Costa Rica,] Ministerio de Salud de Costa Rica. website.
- [Municipalidad de Pérez Zeledón,] Municipalidad de Pérez Zeledón. website.
- [Odoo S.A., 2023] Odoo S.A. (2023). Odoo documentation.
- [Python Software Foundation, 2023] Python Software Foundation (2023). Python documentation.
- [Schwaber and Sutherland, 2020] Schwaber, K. and Sutherland, J. (2020). The scrum guide.